Design and Implementation of a Lightweight Database Management System with B+ Tree Indexing

*CS 432 Database Systems - Assignment 2

Tejas Lohia
Computer Science and Engineering
Indian Institute of Technology Gandhinagar
Gandhinagar, India
tejas.lohia@iitgn.ac.in

Umang Shikarvar
Computer Science and Engineering
Indian Institute of Technology Gandhinagar
Gandhinagar, India
umang.shikarvar@iitgn.ac.in

Zainab Kapadia
Computer Science and Engineering
Indian Institute of Technology Gandhinagar
Gandhinagar, India
zainab.kapadia@iitgn.ac.in

Siddharth Rajandekar
Computer Science and Engineering
Indian Institute of Technology Gandhinagar
Gandhinagar, India
siddharth.rajandekar@iitgn.ac.in

Rishabh Jogani
Computer Science and Engineering
Indian Institute of Technology Gandhinagar
Gandhinagar, India
rishabh.jogani@iitgn.ac.in

***Abstract*—Database indexing is critical for efficient data retrieval in modern database management systems. This paper presents the design and implementation of a lightweight, in-memory database management system (DBMS) featuring B+ tree indexing for optimized query performance. The system implements core database operations including insertion, deletion, update, and range queries while maintaining logarithmic time complexity for most operations. We developed a complete B+ tree index structure with node splitting, merging, and balancing mechanisms to ensure optimal performance. A comprehensive performance analysis comparing the B+ tree implementation against a linear search baseline demonstrates significant performance improvements, with the B+ tree achieving up to 194x speedup for insertion operations, 71x speedup for search operations, and 36x speedup for deletion operations on datasets containing 5,000 records. The system is implemented in Python and provides a clean API for table management and query execution, making it suitable for educational purposes and lightweight database applications.**

***Index Terms*—Database management systems, B+ tree, indexing, data structures, query optimization, performance analysis**

I. Introduction

Database management systems (DBMS) are foundational components of modern computing infrastructure, enabling efficient storage, retrieval, and manipulation of large volumes of data. The performance of a DBMS is heavily dependent on its indexing strategy, which determines how quickly data can be located and accessed [1].

A. Motivation

Traditional database systems rely on efficient indexing structures to provide sub-linear query performance. Among various indexing techniques, the B+ tree has emerged as the de facto standard for database indexing due to its balanced structure, efficient range query support, and predictable performance characteristics [2]. Understanding the implementation and performance characteristics of B+ trees is essential for database practitioners and researchers.

B. Problem Statement

This project addresses the need for an implementation of a lightweight DBMS with B+ tree indexing. Specifically, we aim to:

- Design and implement a complete B+ tree data structure supporting all fundamental operations
- Develop a table abstraction layer providing database-like functionality
- Create a database manager capable of handling multiple tables
- Conduct comprehensive performance analysis comparing indexed vs. non-indexed approaches
- Provide empirical evidence of B+ tree performance characteristics

C. Contributions

The main contributions of this work include:

1) A complete, production-quality implementation of B+ tree indexing with configurable order parameter
2) Support for all core database operations: INSERT, SELECT, UPDATE, DELETE, and RANGE queries
3) A modular architecture separating index structure, table abstraction, and database management
4) Comprehensive performance benchmarking framework comparing B+ tree against linear search baseline
5) Detailed analysis of time and space complexity for various operations
6) Visualization capabilities for understanding tree structure and evolution

D. Organization

The remainder of this paper is organized as follows: Section II reviews related work and background on database indexing. Section III presents the system architecture and design decisions. Section IV details the implementation of key components. Section V presents comprehensive performance analysis and experimental results. Section VI discusses findings and limitations. Section VII concludes and outlines future work.

II. Background and Related Work

A. Database Indexing

Indexing is a database optimization technique that allows for faster retrieval of records. Without an index, a database must scan every row in a table to find matching records, resulting in O(n) time complexity. With proper indexing, search operations can be reduced to O(log n) or even O(1) in optimal cases [3].

B. B+ Tree Structure

The B+ tree is a self-balancing tree data structure that maintains sorted data and allows searches, sequential access, insertions, and deletions in logarithmic time. Key characteristics include:

- All data records are stored in leaf nodes
- Internal nodes contain only keys for navigation
- Leaf nodes are linked to support efficient range queries
- The tree remains balanced through split and merge operations
- Each node contains between $\lceil m/2 \rceil$ and m children, where m is the order

The B+ tree variant, which stores all data in leaf nodes with internal nodes serving purely as indexes, has become the preferred choice for database systems [4].

C. Comparison with Other Index Structures

Several index structures have been proposed for database systems:

- **Hash Indexes**: Provide O(1) average-case search but do not support range queries
- **Binary Search Trees**: Simple but can become unbalanced, degrading to O(n) in worst case
- **AVL Trees**: Maintain strict balance but require more rotations than B+ trees
- **Skip Lists**: Probabilistic alternative with similar performance but simpler implementation

B+ trees are preferred in database systems because they:

1) Minimize disk I/O by allowing large node sizes
2) Support efficient range queries through leaf-level linking
3) Maintain guaranteed logarithmic performance through balancing
4) Provide sequential access patterns beneficial for magnetic storage

D. Modern Database Systems

Most production database systems utilize B+ tree variants for indexing:

- **PostgreSQL**: Uses B+ trees for its primary indexing mechanism
- **MySQL/InnoDB**: Employs clustered B+ tree indexes
- **Oracle Database**: Uses B*-trees (a variant of B+ trees)
- **SQLite**: Implements B-tree and B+ tree indexes

Recent research has focused on optimizing B+ trees for modern hardware, including cache-conscious designs [5] and write-optimized variants [6].

III. System Architecture

A. Overview

The system architecture follows a layered approach with clear separation of concerns. The architecture consists of four primary layers:

1) **Index Layer**: Core B+ tree implementation
2) **Table Layer**: Database table abstraction
3) **Management Layer**: Multi-table database manager
4) **Performance Layer**: Benchmarking and analysis tools

B. Design Principles

The system design adheres to several key principles:

- **Modularity**: Each component has a single, well-defined responsibility
- **Extensibility**: New features can be added without modifying existing code
- **Testability**: Components can be tested independently
- **Performance**: Efficient algorithms and data structures throughout
- **Clarity**: Clean, readable code with comprehensive documentation

C. Component Interactions

The Database Manager serves as the entry point, managing multiple Table instances. Each Table wraps a B+ Tree index, providing a database-like interface. The Performance Analyzer operates independently, comparing B+ Tree performance against a BruteForceDB baseline.

D. Data Flow

User operations flow through the following sequence:

1) Client invokes operation on DatabaseManager
2) DatabaseManager routes request to appropriate Table
3) Table forwards operation to underlying B+ Tree
4) B+ Tree executes operation and returns result
5) Result propagates back through layers to client

IV. Implementation Details

A. B+ Tree Implementation

1) Node Structure: The B+ tree is implemented using a node-based structure. Each node contains:

```
@dataclass
class BPlusTreeNode:
    leaf: bool
    keys: list[int]
    children: list["BPlusTreeNode"]
    values: list[Any]
    next: "BPlusTreeNode | None"
```

Listing 1: B+ Tree Node Structure

Internal nodes use the `keys` and `children` attributes for navigation, while leaf nodes use `keys`, `values`, and `next` for data storage and sequential access.

2) Insertion Algorithm: The insertion operation follows these steps:

1) Check root capacity (using `max_keys` for leaf nodes, `order` for internal nodes)
2) If root is full, split it and create new root
3) Recursively find appropriate leaf node
4) Before descending to a child, check if child is full and split proactively if needed
5) Insert key-value pair in sorted order
6) Refresh separator keys in internal nodes

A key implementation detail is that child nodes are split *before* recursion rather than after, preventing the need for backtracking when a child becomes full during insertion. Additionally, the capacity check distinguishes between leaf nodes (limited by `order - 1` keys) and internal nodes (limited by `order` keys).

```
def insert(self, key: int, value: Any):
    root_max = self.max_keys if self.root.leaf else
    self.order
    if len(self.root.keys) == root_max:
        old_root = self.root
        self.root = BPlusTreeNode(
            leaf=False,
            children=[old_root]
        )
        self._split_child(self.root, 0)

    self._insert_non_full(self.root, key, value)
```

Listing 2: B+ Tree Insertion (Simplified)

3) Node Splitting: The `_split_child` function handles node overflow by splitting a full child node. The implementation uses different mid-point calculations for leaf and internal nodes:

```
def _split_child(self, parent, index):
    child = parent.children[index]
    new_sibling = BPlusTreeNode(leaf=child.leaf)

    if child.leaf:
        mid = len(child.keys) // 2
        new_sibling.keys = child.keys[mid:]
        new_sibling.values = child.values[mid:]
        child.keys = child.keys[:mid]
        child.values = child.values[:mid]

        new_sibling.next = child.next
        child.next = new_sibling

        parent.keys.insert(index,
                           new_sibling.keys[0])
        parent.children.insert(index + 1,
                               new_sibling)
        return

    mid = (self.order + 1) // 2 - 1
    promoted = child.keys[mid]
    new_sibling.keys = child.keys[mid + 1:]
    new_sibling.children = child.children[mid + 1:]

    child.keys = child.keys[:mid]
    child.children = child.children[:mid + 1]

    parent.keys.insert(index, promoted)
    parent.children.insert(index + 1, new_sibling)
```

Listing 3: Node Splitting Logic

For leaf nodes, the split occurs at the midpoint, and the first key of the new sibling becomes the separator. For internal nodes, the mid-point is calculated as $\lfloor (order+1)/2 \rfloor - 1$, and the key at this position is promoted to the parent without being copied to either child.

4) Deletion Algorithm: Deletion maintains tree balance through a two-phase approach:

1) Locate and remove key from leaf node
2) Ensure index bounds are valid after deletion (using `min(idx, len(node.children) - 1)`)
3) If node underflows, attempt to borrow from sibling
4) If borrowing fails, merge with sibling
5) After structural changes, rebuild affected child's separators if it's an internal node
6) Rebuild current node's separator keys from actual first keys of subtrees
7) Return new minimum key for parent updates

The implementation includes three strategies for handling underflow:

- **Borrow from left sibling**: Move largest key from left sibling
- **Borrow from right sibling**: Move smallest key from right sibling
- **Merge**: Combine node with sibling when borrowing is impossible

A critical implementation detail is that after `_fill_child` completes, the affected child's separators

are rebuilt if it's an internal node. This ensures correctness when merges combine internal nodes, as the merged node's separator keys must accurately reflect the new subtree structure.

```
def _merge(self, node, index):
    left = node.children[index]
    right = node.children[index + 1]

    if left.leaf:
        left.keys.extend(right.keys)
        left.values.extend(right.values)
        left.next = right.next
    else:
        # Push down parent separator
        left.keys.append(node.keys[index])
        left.keys.extend(right.keys)
        left.children.extend(right.children)

    del node.keys[index]
    del node.children[index + 1]
```

Listing 4: Node Merging

For internal node merges, the separator key from the parent is pushed down into the merged node. This is essential to maintain the B+ tree invariant that an internal node with $n+1$ children must have exactly n separator keys, where `keys[i]` equals the first key of `children[i+1]`. When merging two internal nodes with children lists C_L and C_R, the combined child list creates a new adjacent pair at the boundary, requiring the parent's separator key to properly divide the merged key space.

```
def _delete(self, node, key):
    if node.leaf:
        idx = bisect_left(node.keys, key)
        if idx >= len(node.keys) or
           node.keys[idx] != key:
            return False, None
        del node.keys[idx]
        del node.values[idx]
        new_min = node.keys[0] if node.keys
                  else None
        return True, new_min

    idx = bisect_right(node.keys, key)
    deleted, _ = self._delete(
        node.children[idx], key)
    if not deleted:
        return False, None

    idx = min(idx, len(node.children) - 1)

    child_node = node.children[idx]
    if child_node.leaf:
        underflows = len(child_node.keys) <
                     self._min_keys(child_node)
    else:
        underflows = len(child_node.children) <
                     ceil(self.order / 2)
    if underflows:
        num_children_before = len(node.children)
        self._fill_child(node, idx)
        merge_happened =
            len(node.children) < num_children_before

        # Determine affected child after merge
        if merge_happened:
            affected_idx = idx - 1 if idx >=
                len(node.children) else idx
        else:
            affected_idx = idx

        # Rebuild affected child's separators
        affected = node.children[affected_idx]
        if not affected.leaf:
            affected.keys = [
                self._first_key(
                    affected.children[i + 1])
                for i in range(
                    len(affected.children) - 1)
            ]

    # Rebuild current node's separators
    node.keys = [
        self._first_key(node.children[i + 1])
        for i in range(len(node.children) - 1)
    ]

    new_min = self._first_key(node.children[0])
              if node.children else None
    return True, new_min
```

Listing 5: Deletion with Underflow Handling

The inclusion of `idx = min(idx, len(node.children) - 1)` ensures that the index remains valid after recursive deletion, preventing out-of-bounds access when keys are removed from the key array. The deletion returns a tuple `(deleted, new_min)` where `new_min` tracks the new minimum key of the subtree for parent separator updates.

5) Separator Key Management: The implementation employs a two-level approach to separator key management:

1) **Local rebuild**: After `_fill_child` completes during deletion, if the affected child is an internal node, its separators are immediately rebuilt from its children
2) **Current node rebuild**: The current node's separators are rebuilt after all structural changes complete

This ensures separator consistency without requiring full tree traversal:

```
# After fill_child completes
if not affected.leaf:
    affected.keys = [
        self._first_key(affected.children[i+1])
        for i in range(len(affected.children)-1)
    ]

# Rebuild current node's separators
node.keys = [
    self._first_key(node.children[i+1])
    for i in range(len(node.children)-1)
]
```

Listing 6: Inline Separator Rebuild in Delete

The `_first_key` helper traverses to the leftmost leaf of a subtree to retrieve its minimum key:

```
def _first_key(self, node):
    cursor = node
    while not cursor.leaf:
        cursor = cursor.children[0]
    if not cursor.keys:
        raise ValueError("Empty leaf encountered")
    return cursor.keys[0]
```

Listing 7: First Key Helper

This approach maintains O(order) cost per level during deletion, resulting in O(order × log n) total complexity rather than O(n) if full tree traversal were used.

6) Search and Range Queries: Point searches traverse from root to appropriate leaf:

```
def search(self, key: int) -> Any | None:
    leaf = self._find_leaf(key)
    idx = bisect_left(leaf.keys, key)
    if idx < len(leaf.keys) and
       leaf.keys[idx] == key:
        return leaf.values[idx]
    return None
```

Listing 8: B+ Tree Search

Range queries exploit the leaf-level linked list:

```
def range_query(self, start_key: int,
                end_key: int) -> list:
    result = []
    leaf = self._find_leaf(start_key)

    while leaf is not None:
        for key, value in zip(leaf.keys,
                              leaf.values):
            if key < start_key:
                continue
            if key > end_key:
                return result
            result.append((key, value))
        leaf = leaf.next

    return result
```

Listing 9: B+ Tree Range Query

This implementation provides O(log n + k) time complexity, where k is the number of results.

B. Table Abstraction

The Table class provides a database-style interface over the B+ tree:

```
class Table:
    def __init__(self, name: str, order: int = 4):
        self.name = name
        self.index = BPlusTree(order=order)

    def insert(self, key: int, record: Any):
        self.index.insert(key, record)

    def select(self, key: int) -> Any | None:
        return self.index.search(key)

    def range_select(self, start: int, end: int):
        return self.index.range_query(start, end)
```

Listing 10: Table Implementation

This abstraction enables database-like operations while maintaining the performance benefits of B+ tree indexing.

C. Database Manager

The DatabaseManager coordinates multiple tables:

```
class DatabaseManager:
    def __init__(self):
        self._tables: dict[str, Table] = {}

    def create_table(self, name: str,
                     order: int = 4) -> Table:
        if name in self._tables:
            raise ValueError(f"Table exists")
        table = Table(name=name, order=order)
        self._tables[name] = table
        return table

    def get_table(self, name: str) -> Table:
        if name not in self._tables:
            raise KeyError(f"Table not found")
        return self._tables[name]
```

Listing 11: Database Manager

D. Performance Analysis Framework

The PerformanceAnalyzer benchmarks B+ tree against a linear search baseline:

```
class PerformanceAnalyzer:
    def run(self, sizes: list[int]) -> list:
        results = []
        for size in sizes:
            # Generate test data
            keys = list(range(size))
            random.shuffle(keys)

            # Benchmark B+ Tree
            bptree_result = self._benchmark(
                BPlusTree(order=self.order),
                keys
            )

            # Benchmark brute force
            brute_result = self._benchmark(
                BruteForceDB(),
                keys
            )

            results.extend([
                bptree_result,
                brute_result
            ])

        return results
```

Listing 12: Performance Benchmarking

The framework measures:

- Insertion time for bulk loads
- Search time for random lookups
- Deletion time for random removals
- Range query performance
- Peak memory consumption

V. Performance Analysis

A. Experimental Setup

1) Hardware Configuration:

- Processor: Apple M2 or equivalent
- Memory: 8GB RAM
- Operating System: macOS/Linux
- Python Version: 3.10+

2) Test Parameters:

- Dataset sizes: 100, 1000, 5000 records
- B+ tree order: 16
- Number of search operations: 200 per size
- Number of delete operations: 100 per size
- Number of range queries: 100 per size
- Random seed: 42 (for reproducibility)

B. Experimental Results

1) Insertion Performance: Table I shows insertion performance comparison:

TABLE I: Insertion Time Comparison (milliseconds)

Size	B+ Tree	Brute Force	Speedup
100	0.48	0.45	0.94x
1000	7.12	78.29	11.00x
5000	15.31	2970.58	194.03x

The B+ tree demonstrates superior insertion performance as dataset size increases. At small sizes (100 records), the overhead of tree maintenance causes slightly slower insertion than brute force. However, as data grows, the logarithmic complexity O(log n) of B+ tree insertion dramatically outperforms the O(n) complexity of the brute force approach, achieving 194x speedup at 5000 records.

2) Search Performance: Table II presents search operation results:

TABLE II: Search Time Comparison (milliseconds)

Size	B+ Tree	Brute Force	Speedup
100	0.13	0.15	1.12x
1000	0.14	1.73	12.76x
5000	0.12	8.50	71.43x

Search performance demonstrates the most significant advantage of B+ tree indexing. The B+ tree maintains nearly constant search time across all dataset sizes (0.12–0.14 ms), while brute force search time grows linearly. For 5000 records, the B+ tree achieves over 71x speedup, validating the O(log n) vs O(n) complexity difference.

3) Range Query Performance: Table III shows range query results:

TABLE III: Range Query Time Comparison (milliseconds)

Size	B+ Tree	Brute Force	Speedup
100	2.22	0.82	0.37x
1000	7.28	6.38	0.88x
5000	54.27	91.55	1.69x

Range query performance shows interesting characteristics. At smaller dataset sizes, the brute force approach is faster due to simpler iteration without tree traversal overhead. However, as data grows, the B+ tree's leaf-level linked list becomes advantageous, achieving 1.69x speedup at 5000 records. The crossover point occurs around 1000 records, after which B+ tree performance scales better.

4) Deletion Performance: Table IV shows deletion operation results:

TABLE IV: Deletion Time Comparison (milliseconds)

Size	B+ Tree	Brute Force	Speedup
100	2.30	0.18	0.08x
1000	1.45	6.47	4.47x
5000	1.75	62.59	35.77x

Deletion performance reveals the cost of maintaining B+ tree balance. At small sizes, the overhead of node merging and separator rebuilding makes B+ tree deletion slower. However, as data grows, the logarithmic complexity dominates, achieving 35x speedup at 5000 records. The implementation uses O(order $\times$ log n) separator rebuilding after structural changes to maintain correctness.

5) Memory Consumption: Table V compares memory usage:

TABLE V: Peak Memory Consumption (KB)

Size	B+ Tree	Brute Force	Overhead
100	15.19	6.61	2.30x
1000	289.13	280.36	1.03x
5000	1702.65	1629.16	1.05x

The B+ tree incurs higher memory overhead at small sizes (2.3x) due to node structure and pointer storage. However, this overhead diminishes significantly as dataset size grows, converging to only 3–5% overhead at larger sizes. This modest overhead is acceptable given the substantial performance improvements for search and insertion operations.

C. Complexity Analysis

1) Theoretical Complexity: Table VI summarizes theoretical complexity:

TABLE VI: Time Complexity Comparison

Operation	B+ Tree	Brute Force
Insert	O(log n)	O(n)
Search	O(log n)	O(n)
Delete	O(order $\times$ log n)	O(n)
Update	O(log n)	O(n)
Range Query	O(log n + k)	O(n)

Where n is the number of records, k is the result set size for range queries, and order is the B+ tree branching factor (a small constant, typically 4–100). The delete operation has O(order $\times$ log n) complexity due to separator key rebuilding at each level after structural modifications.

2) Empirical Validation: The experimental results validate theoretical complexity predictions:

- B+ tree search time remains nearly constant (0.12–0.14 ms) across all sizes, confirming O(log n) behavior
- Brute force operations show clear linear growth: search time increases from 0.15 ms to 8.50 ms (57x) as size grows from 100 to 5000 (50x)

- Insertion speedup increases from 0.94x to 194x, demonstrating that B+ tree overhead is amortized at scale
- The performance gap widens exponentially as dataset size increases

VI. Discussion

A. Key Findings

The experimental analysis reveals several important findings:

1) **Consistent Performance**: B+ tree provides predictable, logarithmic performance across insertion and search operations
2) **Scalability**: Performance advantage increases dramatically with dataset size—194x speedup for insertion at 5000 records
3) **Search Excellence**: Search operations maintain near-constant time (0.12–0.14 ms) regardless of dataset size
4) **Memory Efficiency**: Memory overhead diminishes from 2.3x at small sizes to just 3–5% at larger sizes
5) **Range Query Crossover**: B+ tree range queries become advantageous beyond approximately 1000 records

B. Implementation Quality

The implementation demonstrates several strengths:

- **Correctness**: All operations maintain B+ tree invariants
- **Completeness**: Full support for insert, delete, update, search, and range queries
- **Modularity**: Clean separation between index, table, and database layers
- **Testability**: Comprehensive smoke tests validate functionality

C. Limitations

Several limitations should be acknowledged:

1) **In-Memory Only**: Current implementation does not support persistence
2) **Integer Keys**: Only integer keys are supported
3) **Single-Column Index**: No support for composite indexes
4) **No Concurrency**: Not thread-safe or concurrent
5) **No Transactions**: No ACID properties or transaction support

D. Design Trade-offs

Several design decisions involved trade-offs:

- **Order Parameter**: Configurable order allows tuning for different workloads but adds complexity
- **Separator Rebuild Strategy**: Rebuilding separator keys after structural changes ensures correctness with O(order $\times$ log n) cost rather than attempting incremental updates that are error-prone
- **Python Implementation**: Prioritizes clarity over raw performance
- **Two-Level Separator Refresh**: After merges, both the affected child and current node rebuild their separators, trading some performance for guaranteed consistency

E. Real-World Applicability

While this is an educational implementation, the architecture and algorithms align with production database systems:

- Similar B+ tree structure to PostgreSQL and MySQL
- Comparable node splitting and merging strategies
- Standard approach to range query optimization

The main differences from production systems involve:

- Disk-based storage with page management
- Concurrency control and locking
- Write-ahead logging for durability
- Query optimization and execution planning

VII. Conclusion and Future Work

A. Summary

This project successfully designed and implemented a lightweight database management system with B+ tree indexing. The implementation demonstrates:

- Complete B+ tree with all fundamental operations
- Clean, modular architecture
- Significant performance improvements over linear search
- Empirical validation of theoretical complexity analysis

Performance analysis confirms that B+ tree indexing provides substantial benefits, with insertion operations showing up to 194x speedup, search operations achieving 71x speedup, and deletion operations demonstrating 36x speedup on 5000 record datasets. The implementation serves as both an educational tool for understanding database indexing and a functional lightweight DBMS for small-scale applications.

B. Future Enhancements

Several directions for future work include:

1) Persistence Layer:

- Implement page-based storage on disk
- Add buffer pool manager for caching
- Develop serialization/deserialization mechanisms
- Support database file format

2) Advanced Indexing:

- Support for composite (multi-column) indexes
- String and floating-point key types
- Hash indexes for exact-match queries
- Full-text search indexes

3) Query Processing:

- SQL parser and query planner
- Query optimization with cost estimation
- Join algorithms (nested loop, hash join, merge join)
- Aggregation and sorting operations

4) Concurrency and Recovery:

- Multi-version concurrency control (MVCC)
- Lock-based concurrency control
- Write-ahead logging (WAL)
- Crash recovery mechanisms

5) Performance Optimizations:

- Cache-conscious data structures
- SIMD-based operations
- Lock-free data structures
- Adaptive indexing

C. Educational Impact

This implementation provides a valuable educational resource for understanding:

- Database indexing fundamentals
- B+ tree algorithms and data structures
- Performance analysis methodologies
- System design principles

The clean, well-documented code serves as a reference for students learning database internals.

D. Final Remarks

The successful implementation of this lightweight DBMS with B+ tree indexing demonstrates the power of well-designed data structures in achieving efficient data management. The significant performance improvements observed validate decades of research in database indexing and highlight why B+ trees remain the indexing structure of choice in modern database systems.

REFERENCES

[1] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. New York, NY: McGraw-Hill, 2003.

[2] D. Comer, "Ubiquitous B-tree," *ACM Computing Surveys (CSUR)*, vol. 11, no. 2, pp. 121-137, 1979.

[3] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 6th ed. New York, NY: McGraw-Hill, 2010.

[4] G. Graefe, "Modern B-tree techniques," *Foundations and Trends in Databases*, vol. 3, no. 4, pp. 203-402, 2011.

[5] J. Rao and K. A. Ross, "Making B+-trees cache conscious in main memory," in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 2000, pp. 475-486.

[6] M. A. Bender, M. Farach-Colton, W. Jannen, R. Johnson, B. C. Kuszmaul, D. E. Porter, J. Yuan, and Y. Zhan, "An introduction to B^ϵ-trees and write-optimization," *;login:*, vol. 40, no. 5, pp. 22-28, 2015.

[7] R. Garcia-Molina, J. D. Ullman, and J. Widom, *Database Systems: The Complete Book*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2008.

[8] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Reading, MA: Addison-Wesley, 1998.

[9] SQLite Development Team, "SQLite Database File Format," 2023. [Online]. Available: https://www.sqlite.org/fileformat.html

[10] PostgreSQL Global Development Group, "PostgreSQL Documentation: Index Types," 2023. [Online]. Available: https://www.postgresql.org/docs/current/indexes-types.html